Comprehensive Architectural Framework for a Secure Loopback Proxy: Integrating Coraza WAF, Kernel Killswitches, and Cryptographic Authentication

The contemporary cybersecurity landscape dictates that network perimeters are no longer absolute boundaries of trust. Historically, localized interfaces—such as the loopback address space (127.0.0.1 or localhost)—were treated as implicitly secure enclaves, operating under the assumption that internal communication channels were immune to external manipulation. However, the proliferation of sophisticated Server-Side Request Forgery (SSRF) and DNS rebinding attacks has unequivocally demonstrated that public-facing vulnerabilities can be leveraged to pivot into local environments. To establish a true zero-trust architecture on a local machine, security engineers must implement rigorous application-layer inspection, state-of-the-art cryptographic access controls, and kernel-level network enforcement directly on the loopback interface.

This report delineates an exhaustive architectural framework for deploying and securing a local proxy operating on port 1137. The proposed system utilizes a Go-based reverse proxy fortified by the Coraza Web Application Firewall (WAF) implementing the OWASP Core Rule Set (CRS). To ensure resilience against volumetric and targeted attacks, the architecture integrates advanced Denial-of-Service (DoS) mitigations, brute-force countermeasures, and a dynamic, kernel-level nftables killswitch. Access to the underlying proxy routes is strictly gated by a Single Page Application (SPA) requiring cryptographic password verification utilizing the Argon2id hashing algorithm. Upon successful authentication, the system provisions a strictly validated JSON Web Token (JWT) with a 24-hour expiration and serves a user-selected ASCII art interface, visually confirming the establishment of the secure boundary. The subsequent sections provide a granular technical deconstruction of this architecture, substantiated by the latest 2025 and 2026 academic research, cryptographic standards, and vulnerability analyses.

Proxy Engine Architecture and Loopback Network Isolation

The foundational component of this architecture is a high-performance reverse proxy written in the Go programming language, bound exclusively to the local loopback interface on port 1137. By leveraging Go's native httputil.ReverseProxy package, the system inherits highly optimized connection pooling, goroutine-based concurrency for multiplexed client handling, and efficient streaming I/O capabilities[1]. However, binding a service to the loopback interface introduces unique attack vectors that must be mitigated at the application layer.

The Mechanics of DNS Rebinding on Local Interfaces

DNS rebinding attacks have emerged as a primary vector for exploiting loopback-bound services, circumventing the protections typically afforded by network firewalls. In a standard DNS rebinding execution, an adversary lures a victim to a malicious website hosted on an attacker-controlled public domain. The authoritative DNS server for this domain initially responds with a legitimate public IP address, satisfying the browser's initial connection request. Once the page is loaded within the victim's browser, the attacker rapidly alters the DNS record to resolve to 127.0.0.1, while configuring the DNS response with an exceptionally short Time-to-Live (TTL)[2].

Subsequent asynchronous requests initiated by malicious JavaScript on the page (such as fetch or XMLHttpRequest calls) prompt the browser to perform a new DNS lookup due to the expired TTL. The domain now resolves to the victim's local loopback interface. Because the browser relies on the hostname to enforce the Same-Origin Policy (SOP), it permits the cross-origin request, effectively allowing the public webpage to interact directly with the local proxy running on port 1137[4].

Recent vulnerability disclosures from 2026 underscore the catastrophic impact of missing origin validation on local interfaces. For example, CVE-2026-77087 detailed a remote code execution flaw in the Paperclip developer tool, which exposed an HTTP API on localhost. Attackers utilized DNS rebinding to execute arbitrary operating system commands on developer workstations because the local server implicitly trusted all loopback traffic and failed to validate the Host header[2]. Similarly, CVE-2026-64849 exposed an unauthenticated full-read SSRF vulnerability in the MLflow tracking server. The root cause was a Time-of-Check to Time-of-Use (TOCTOU) race condition in the SSRF guard. The application validated a webhook URL via socket.getaddrinfo() to ensure it was a globally routable IP, but discarded the resolved IP. The subsequent HTTP request independently re-resolved the hostname, creating a window for DNS rebinding where the attacker's DNS server could return a private or link-local IP (such as the AWS metadata endpoint at 169.254.169.254) during the actual request execution[5].

To conclusively neutralize DNS rebinding at the proxy layer, the Go implementation must rigidly enforce Host header validation prior to any further processing. The proxy on port 1137 is programmed to immediately drop any incoming HTTP request where the Host header does not strictly match localhost:1137 or 127.0.0.1:1137. This strict origin-pinning severs the DNS rebinding attack chain. When the browser executes the rebound request, it appends the attacker's public hostname to the Host header; the proxy detects the mismatch and terminates the TCP connection, preventing the malicious payload from reaching the Coraza WAF or the authentication gate[3].

Transport Layer Security (TLS 1.3) and Cryptographic Infrastructure

Despite operating entirely on a local interface, the proxy must enforce Transport Layer Security (TLS) to ensure that sensitive credentials, cryptographic hashes, and session tokens are protected from local packet sniffing and cross-process memory inspection. The architecture exclusively mandates TLS version 1.3, which offers profound cryptographic enhancements over

its predecessors. TLS 1.3 introduces a streamlined 1-RTT (Round Trip Time) handshake, deprecates vulnerable cipher suites (such as RSA key transport in favor of forward-secret Ephemeral Elliptic Curve Diffie-Hellman), and encrypts the Server Name Indication (SNI) extension[6].

Certificate Generation and the Subject Alternative Name Requirement

To implement TLS 1.3 on localhost, a self-signed certificate must be generated and trusted by the local operating system. A prevalent architectural flaw in modern TLS deployments is the generation of certificates relying solely on the Common Name (CN) attribute. Modern TLS clients, including standard web browsers and the Go crypto/tls stack, strictly enforce the RFC 2818 requirement that hostnames must be validated against the Subject Alternative Name (SAN) extension. A certificate lacking a correctly formatted SAN will result in an immediate ERR_TLS_CERT_ALTNAME_INVALID error, abruptly aborting the cryptographic handshake during the Client Verification phase[7].

The certificate for the port 1137 proxy is generated utilizing OpenSSL 3.x, enforcing modern cryptographic primitives. The generation command explicitly injects the required SAN parameters while utilizing the Ed25519 elliptic curve. Ed25519 is prioritized because it ensures highly efficient signature generation and verification while maintaining strict compatibility with TLS 1.3 cipher suites[10].

```Bash
openssl req -x509 -nodes -newkey ed25519 -keyout localhost.key -out localhost.crt -days 365 -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

TLS Handshake Mechanics and Go Configuration

The TLS 1.3 handshake represents a paradigm shift in connection security. During the initial ClientHello, the client transmits its supported TLS versions, a list of acceptable cipher suites, a random value, and, crucially, its key exchange parameters. In the subsequent ServerHello, the server selects the protocol version and cipher suite, sends its own random value and key exchange parameters, and transmits its certificate chain[9]. Upon verifying the certificate's signature chain and SAN matching, both parties independently derive identical session keys utilizing the exchanged material. No symmetric key is ever transmitted across the wire, ensuring that an eavesdropper capturing the entire handshake cannot compute the session keys without the ephemeral private key material[9].

To enforce this standard within the Go proxy configuration, the tls.Config struct is explicitly configured with the MinVersion attribute set to tls.VersionTLS13 and the MaxVersion similarly constrained. This explicit parameterization completely neutralizes protocol downgrade attacks, ensuring that clients cannot force the proxy to fall back to vulnerable TLS 1.2 cipher suites[7].

Web Application Firewall Integration: Coraza and OWASP CRS

Before any decrypted HTTP request reaches the Single Page Application authentication gate or the backend service, it must traverse a robust, Layer 7 deep-packet inspection layer. For this proxy architecture, the Coraza Web Application Firewall is embedded directly into the Go application logic as an http.Handler middleware[11].

The Coraza Engine Architecture

Coraza is an enterprise-grade, memory-safe WAF engine engineered entirely in Go. It serves as a modern, high-performance successor to the legacy ModSecurity engine, which relied heavily on C bindings (libmodsecurity) and frequently suffered from memory safety vulnerabilities and deployment complexities[13]. Coraza processes HTTP transactions by executing security evaluations across four distinct operational phases: Request Headers (Phase 1), Request Body (Phase 2), Response Headers (Phase 3), and Response Body (Phase 4)[15]. By natively integrating Coraza into the Go reverse proxy via the tx.ProcessRequest() and tx.ProcessResponseBody() hooks, the proxy architecture circumvents the latency overhead, context-switching, and memory fragmentation inherently associated with sidecar deployments or WebAssembly (WASM) proxy filters[11].

Memory footprint optimization is a paramount concern for loopback proxies designed for high concurrency. Extensive load testing and profiling from industry deployments in 2026 demonstrate that a compiled Coraza instance running the full OWASP Core Rule Set consumes approximately 25MB of baseline RAM per instance. This memory is predominantly allocated to compiled regular expression trees and Aho-Corasick pattern matchers, which account for roughly 43% of the memory cost[17]. To prevent resource exhaustion attacks wherein an adversary sends massive payloads to consume WAF processing memory, the engine must be configured with a strict SecRequestBodyLimit. Empirical research indicates that buffering payloads larger than 4KB in memory for inspection significantly increases garbage collection latency and elevates the risk of Out-of-Memory (OOM) crashes. Establishing a 4KB body inspection limit with SecRequestBodyLimitAction ProcessPartial provides an optimal equilibrium, allowing the WAF to identify embedded malicious payloads without compromising the stability of the underlying proxy[17].

OWASP Core Rule Set (CRS) v4 Implementation

Coraza relies on the OWASP Core Rule Set (CRS) version 4.x to provide comprehensive, signature-based and anomaly-based defense mechanisms. The CRS is a community-maintained collection of generic attack detection rules designed to mitigate the OWASP Top 10 vulnerabilities, including SQL Injection (SQLi), Cross-Site Scripting (XSS), Local File Inclusion (LFI), and Remote Command Execution (RCE)[17].

The modern CRS operates on an anomaly-scoring paradigm rather than immediate, localized blocking. Historically, binary pattern matching resulted in unacceptably high false-positive rates. Under the anomaly-scoring model, each triggered rule contributes a predefined numeric weight to a cumulative transaction score. The request is only disrupted (e.g., returning an HTTP

403 Forbidden status) if the total anomaly score exceeds the globally configured tx.inbound_anomaly_score_threshold, which is typically set to 5 for inbound requests and 4 for outbound responses[19]. The rules are authored in SecLang, featuring a highly structured syntax: SecRule VARIABLES "@OPERATOR OPERATOR_ARGUMENTS" "ACTIONS". For example, a rule targeting SQL injection might parse the ARGS variable utilizing the @detectSQLi operator, appending anomaly points during Phase 2 if a match occurs[19].

Statistical Evaluation of Paranoia Levels

The strictness and sensitivity of the OWASP CRS are governed by the Paranoia Level (PL), ranging from PL1 to PL4. Higher paranoia levels activate increasingly restrictive regular expressions and scrutinize a broader array of HTTP anomalies, but they concurrently elevate the probability of false positives[16]. A comprehensive 2025 study published in the *Journal of Engineering Proceedings* evaluated Coraza's performance running CRS v4.15.0 across different reverse proxy environments (Envoy via WASM versus native Caddy in Go) utilizing a testbed simulating legitimate HTTPS traffic (Eramba web application) and structured OWASP Top-Ten attacks[16].

The statistical results, analyzed via Welch's t-test and Levene's test for variance homogeneity across 30 sequential request blocks, demonstrated the critical trade-offs inherent in Paranoia Level calibration:

Paranoia Level	Envoy Mean False Positives ($\bar{x}$)	Caddy Mean False Positives ($\bar{x}$)	Welch's t-Test (p-value)	Architectural Observation
PL1	0.000	0.000	N/A	Perfectly stable baseline configuration; allowed all legitimate data submission.
PL2	18.567	36.300	1.1×10^{-4}	Envoy demonstrated significantly superior precision; increased sensitivity intercepted complex payloads.

PL3	24.967	36.367	0.0182	Transitional threshold; legitimate POST requests began to trigger anomalies due to strict byte encoding rules.
PL4	138.000	24.933	8.09×10^{-9}	Caddy maintained stability, whereas Envoy became excessively restrictive, blocking nearly all incoming traffic.

For the port 1137 loopback proxy, Paranoia Level 2 (PL2) is designated as the optimal configuration. PL2 aggressively intercepts obfuscated injection attempts and abnormal byte encodings without unnecessarily disrupting the standard JSON payloads utilized by the Single Page Application during the password challenge phase[16].

Mitigating HTTP Request Smuggling and Desynchronization

A paramount capability of the Coraza and CRS v4 integration during Phase 1 (Request Headers) is the detection and mitigation of HTTP Request Smuggling, commonly referred to as HTTP Desync attacks. Smuggling exploits systemic inconsistencies in how chained HTTP processors (such as the frontend proxy and the backend application server) parse request boundaries[25]. By transmitting malformed, conflicting, or obfuscated headers—most notably combinations of Content-Length (CL) and Transfer-Encoding (TE)—an adversary can cause the proxy and the backend to disagree on where a specific HTTP request terminates. This desynchronization allows the attacker to "smuggle" a malicious payload that the backend erroneously appends to the subsequent legitimate user's request[20].

Coraza mitigates this severe vulnerability by strictly normalizing HTTP syntax before payload processing begins. The CRS includes dedicated directives (such as Rule ID 950012) designed to identify CL.TE, TE.CL, and TE.TE variants[20]. The engine automatically drops requests containing both Content-Length and Transfer-Encoding headers, as well as requests containing malformed or chunked encoding violations, ensuring that request boundaries remain absolute

and immutable across the proxy architecture[22].

Advanced Denial-of-Service and Brute Force Mitigations

Protecting the cryptographic authentication gate from brute-force credential stuffing and volumetric Denial-of-Service (DoS) attacks requires a multi-tiered mitigation strategy. Relying exclusively on standard application-layer rate-limiting algorithms is a widespread architectural deficiency that permits sophisticated adversaries to bypass thresholds utilizing burst traffic patterns.

The Token Bucket Boundary Problem

In Go, the standard library for implementing rate limits is golang.org/x/time/rate, which utilizes the token bucket algorithm. This algorithm enables developers to define a sustained rate of acceptable requests alongside a maximum burst capacity, effectively managing typical traffic fluctuations[1]. However, naive implementations of the token bucket algorithm expose the proxy to the "window boundary problem."

If an application defines a rate limit as 100 requests per minute with a burst allowance of 20, an attacker can intentionally weaponize the temporal boundaries of the algorithm. By holding their traffic until the final second of the minute window (e.g., 11:59:59), the attacker can rapidly transmit 20 requests. Immediately upon the clock ticking to 12:00:00, the token bucket replenishes, allowing the attacker to transmit another 20 requests[29]. This effectively results in 40 requests impacting the authentication endpoint within a single second. This high-density burst of traffic can overwhelm computationally expensive cryptographic hashing functions, resulting in a localized CPU exhaustion denial-of-service event[29].

WAF-Layer Sliding Window Tracking

To rectify the limitations of standard token buckets, Coraza is configured with custom SecRule directives that implement an adaptive, sliding-window rate-limiting algorithm. By utilizing ModSecurity's persistent collections (specifically initialized via initcol:ip=%{REMOTE_ADDR}), Coraza persistently tracks the exact timestamp and frequency of requests originating from specific IP addresses across the proxy's lifecycle[15].

The WAF-enforced brute-force mitigation ruleset establishes three distinct, overlapping thresholds:

1. **Global Volumetric Limit:** A strict cap of 300 requests per minute across all endpoints, preventing basic HTTP flooding[15].
2. **Authentication Gate Limit:** A severely constrained limit of 10 requests per minute specifically directed at the SPA's /api/login authentication endpoint, directly throttling credential stuffing attempts[15].
3. **Anomaly Threshold Reduction:** If an IP address generates requests that repeatedly trigger CRS anomaly scores (e.g., initiating SQL injection probing alongside authentication attempts), the rate limit is dynamically contracted, executing a preemptive block before volumetric thresholds are even breached[19].

When an IP exceeds these established thresholds, Coraza issues an immediate HTTP 429 Too

Many Requests or HTTP 403 Forbidden response[18]. However, while returning HTTP error codes disrupts the attack, it still requires the Go proxy application to process the TCP handshake, allocate file descriptors, and parse the HTTP headers. During a sustained, highly aggressive brute-force campaign, this application-layer overhead can degrade overall system performance. Absolute isolation requires the implementation of an automated, kernel-level killswitch.

The Kernel-Level Automated Killswitch via nftables

To achieve absolute network isolation against persistent attackers, the architecture features a dynamic killswitch tied directly to the Linux kernel's nftables framework. When the WAF identifies sustained brute-force behavior, it escalates the mitigation from user-space HTTP blocking to kernel-space packet dropping.

nftables Architecture and Policy Routing

nftables is the modern, highly performant successor to iptables within the Linux kernel. It provides superior packet classification, atomic rule replacements, and native support for managing both IPv4 and IPv6 traffic within unified inet tables[31]. Unlike legacy systems that require flushing and reloading the entire firewall ruleset to ban an IP, nftables utilizes dynamic memory sets, allowing for instantaneous IP blacklisting without interrupting existing network connections[33].

The kernel firewall is configured with a fail-closed, default-drop policy on the loopback interface for the proxy port, utilizing packet marking (fwmark) and stateful connection tracking to strictly manage traffic[32]. The foundational nftables ruleset initializes a dynamic set named bruteforce_blacklist configured with a strict 24-hour timeout:

Code snippet

```
table inet proxy_defense {
    set bruteforce_blacklist {
        type ipv4_addr
        flags dynamic, timeout
        timeout 24h
    }

    chain input {
        type filter hook input priority filter; policy accept;
        tcp dport 1337 ip saddr @bruteforce_blacklist counter drop
    }
}
```

Triggering the Killswitch

The integration between the Go reverse proxy and the nftables kernel subsystem forms a closed-loop, automated security architecture. When the Coraza WAF evaluates a request and triggers a brute-force or high-severity anomaly rule (for instance, Rule ID 101801 for excessive authentication failures), it generates a structured JSON audit log entry detailing the offending IP address, the matched rule ID, and the request metadata[35].
A lightweight, concurrent goroutine embedded within the Go proxy continuously monitors this audit stream. Upon detecting a hard violation threshold, the proxy process issues an elevated system call to the nftables command-line utility, injecting the attacker's IP address directly into the bruteforce_blacklist dynamic set:

```bash
nft add element inet proxy_defense bruteforce_blacklist { <Attacker_IP> }
```

The moment the IP address populates the set, the Linux kernel's network stack aggressively drops all subsequent TCP SYN packets originating from that address before they ever reach the user-space Go application[32]. The attacker receives no HTTP error codes, no TCP resets, and the connection attempts simply time out, rendering the attack entirely inert. Furthermore, the timeout 24h flag—natively managed by the nftables kernel module—ensures that the IP address is automatically expunged from the blacklist after exactly 24 hours. This eliminates the need for secondary cron jobs or cleanup scripts, ensuring the killswitch remains performant, automated, and strictly fail-closed[20].

Cryptographic Authentication: The SPA Password Challenge Gate

Legitimate users attempting to traverse the proxy boundary must successfully navigate a Single Page Application (SPA) serving as a password challenge gate. Unlike traditional web applications that rely heavily on server-side rendering, the SPA is a lightweight HTML and JavaScript payload delivered instantaneously upon the initial connection to the root path (/). The SPA prompts the user for their authentication credentials, which are subsequently transmitted securely over the TLS 1.3 encrypted channel to the Go proxy's verification endpoint.

Argon2id: The Apex of Password Hashing

The cryptographic resilience of the password gate is entirely dependent upon the hashing algorithm utilized for credential verification. Legacy algorithms such as MD5, SHA-256, and even PBKDF2 are considered severely compromised in modern threat models because their computation relies solely on CPU cycles. State-sponsored adversaries and sophisticated attackers can rent vast arrays of cloud-based GPUs or deploy Application-Specific Integrated Circuits (ASICs) to execute billions of parallel hash calculations per second, rendering

traditional CPU-bound hashes trivial to crack[36].
To conclusively neutralize hardware-accelerated cracking, the proxy architecture mandates the use of Argon2id, the victor of the 2015 Password Hashing Competition and the algorithm formally standardized for secure password storage in RFC 9106[36]. Argon2id is classified as a "memory-hard" function. This classification indicates that the algorithm forces the underlying hardware to continuously allocate, write to, and read from massive blocks of Random Access Memory (RAM) during the hashing process. Because memory bandwidth and latency present physical bottlenecks for both CPUs and GPUs, adversaries cannot efficiently parallelize Argon2id calculations without incurring astronomical hardware acquisition costs[36].
Argon2id operates as a highly optimized hybrid variant. During the first half of its execution, it functions in a data-independent mode (Argon2i) to provide robust cryptographic resistance against side-channel timing attacks. In the second half, it transitions into a data-dependent memory access mode (Argon2d) to maximize resistance against GPU and ASIC cracking efforts[36]. The algorithm constructs a massive memory matrix $B[i][j]$ consisting of p rows (lanes) and q columns (slices), where blocks are sequentially generated and XORed utilizing the Blake2b compression function, enforcing a rigorous, sequential memory traversal that cannot be optimized away by specialized hardware[43].

Parameter Tuning and Go Implementation

According to the latest 2026 OWASP Password Storage Cheat Sheet and the formal RFC 9106 guidelines, deploying Argon2id requires meticulous tuning of three primary parameters: memory cost (m, defining the size of the memory matrix), time cost or iterations (t, defining the number of passes over the memory), and the degree of parallelism (p, defining the number of independent computational lanes)[36].
For the port 1137 loopback proxy, the OWASP 2026 recommended baseline parameters are strictly enforced:

- **Memory Cost (m):** 19 MiB (19,456 KiB).
- **Time Cost (t):** 2 iterations.
- **Parallelism (p):** 1 lane[36].

Algorithm	Memory Cost	Time Cost (Iterations)	Parallelism	GPU/ASIC Resistance
Argon2id (OWASP 2026 Baseline)	19 MiB	2	1	Extremely High (Memory-Hard)
Scrypt	128 MiB (	8 ($r = 8$)	1	High

(Fallback)	$N = 2^{17}$)			(Memory-Hard)
Bcrypt (Legacy)	N/A (CPU Only)	Cost Factor 14	N/A	Low (Vulnerable to ASICs)
PBKDF2 (Legacy/FIPS)	N/A (CPU Only)	600,000	N/A	Lowest (Vulnerable to GPUs)

These parameters dictate that verifying a single password attempt consumes a non-trivial footprint of RAM, effectively neutralizing massive parallel brute-forcing efforts even before the WAF rate limits are triggered[41].
In Go, verifying the submitted password against the stored Password Hashing Competition (PHC) format string (e.g., $argon2id$v=19$m = 19456, t = 2, p = 1$...) requires extracting the unique salt and parameters, and subsequently re-deriving the key utilizing the golang.org/x/crypto/argon2.IDKey function[44].
A critical, often-overlooked vulnerability in bespoke authentication systems is the reliance on standard string comparison operators (such as == or bytes.Equal) to compare the derived hash against the stored hash. Standard software comparators are designed to short-circuit, returning false the exact millisecond they encounter a mismatched byte. An adversary can meticulously measure the microsecond differences in server response times to determine exactly how many characters of their guessed hash are correct, gradually mapping out the entire secret through a side-channel timing attack[38]. To categorically prevent this, the Go authentication logic employs crypto/subtle.ConstantTimeCompare(). This cryptographic function executes in a fixed, immutable amount of CPU cycles regardless of where the byte divergence occurs, completely masking the comparison outcome from any form of timing analysis[45].

Session Management: Secure JWT Issuance and Transport

Following a successful password challenge, the proxy must issue a session token to maintain the user's authenticated state, thereby avoiding the computationally prohibitive cost of executing Argon2id on every subsequent HTTP request. The architecture utilizes JSON Web Tokens (JWT), standardized under RFC 7519, to achieve stateless, cryptographic session management[47]. To enforce a strict session lifecycle, the token is explicitly minted with an exp (expiration) claim set to exactly 24 hours from the precise time of issuance.

Mitigating "Token Time Bomb" Vulnerabilities

While JWTs are ubiquitous in modern web architectures, their implementation within cryptographic libraries is notoriously fragile. A comprehensive academic evaluation presented at the Network and Distributed System Security (NDSS) Symposium in 2026, detailed in the paper *"Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discovery"*, utilized a novel fuzzing framework named JWTeemo to uncover severe flaws across the ecosystem[49]. By mapping JWT Abstract Syntax Trees (AST) to Function-extended Backus-Naur Form (FBNF) graphs, the researchers identified 31 catastrophic vulnerabilities across 43 popular JWT libraries, analyzing anomaly variations through standard deviation thresholds ($P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}$)[51].

The research identified five primary vulnerability categories in modern JWT processing that this loopback proxy architecture actively defends against:

Vulnerability Category	Root Cause Mechanism	Architectural Mitigation
Algorithm Confusion	Server expects asymmetric signature (e.g., RS256) but accepts symmetric alg header (HS256) utilizing the public key string as the HMAC secret.	Hardcode expected algorithm in Go verification logic; explicitly ignore the client-provided alg header during cryptographic checks[51].
Signature/Encryption Confusion	Library conflates JWS verification keys with JWE decryption keys, allowing forged payload encryption.	Strictly type cryptographic keys; separate signature validation execution from encryption routines[51].
Format Confusion	Library accepts non-standard formats (e.g., JSON-formatted JWS explicitly disallowed by RFC).	Enforce strict RFC compliance; reject any token lacking the exact three-part dot-separated structure[51].
Compression DoS	Library decompresses payload (via zip claim) prior to signature validation, exhausting CPU/Memory.	Reject zip headers entirely; enforce signature validation mathematically *before* any payload decoding occurs[51].
Billion Hashes Attack	CPU resource exhaustion exploiting Password-Based Encryption (PBES2)	Disable PBES2 support within the JWT parsing library configuration[51].

	algorithms within JWEs.	

To strictly adhere to the updated JWT Best Current Practices (RFC 8725) and the mitigations proposed to the IETF by the NDSS 2026 researchers, the Go proxy enforces these rigid validation rules natively, ensuring that forged or computationally expensive tokens are discarded instantaneously[48].

Transport Security: The __Host- Cookie Prefix

A perfectly validated, cryptographically secure JWT is entirely compromised if it is stolen via Cross-Site Scripting (XSS) or intercepted on the network layer. Consequently, the JWT is never returned within a JSON response body for storage in the browser's LocalStorage or accessible document.cookie stores. Instead, the Go proxy issues the JWT within an HttpOnly and Secure cookie, ensuring it cannot be read by client-side JavaScript and is transmitted exclusively over the TLS 1.3 encrypted channel[53].

To further harden the cookie transport mechanism, the architecture utilizes RFC 6265bis cookie prefixes, specifically deploying the __Host- prefix[53]. The HTTP cookie specification contains a fundamental asymmetry: the server issues a highly detailed Set-Cookie header containing attributes like Secure, Path, and SameSite, but the browser only returns the naked name-value pair in subsequent Cookie headers. Consequently, the server has no native way to verify if a returning cookie was actually transmitted securely or if it originated from the correct path[53].

By naming the authentication cookie __Host-SessionToken, the browser natively enforces rigorous security boundaries at the client level before the cookie is even stored:

1. The cookie must have the Secure attribute (prohibiting transmission over unencrypted HTTP).
2. The cookie must not include a Domain attribute (preventing it from being sent to, or overwritten by, adjacent subdomains).
3. The cookie's Path attribute must be set exactly to /[55].

The __Host- prefix provides an ironclad guarantee that the JWT is completely locked to the exact host and origin (https://localhost:1137). Even if an adversary manages to manipulate a subdomain or exploit an adjacent origin, the browser will categorically refuse to overwrite, read, or leak the prefixed cookie, neutralizing advanced session fixation and cookie tossing attacks[53].

SPA Presentation and ASCII Art Rendering

Upon the successful validation of the Argon2id hash and the secure setting of the __Host-prefixed JWT cookie, the server responds with a success status to the Single Page Application. The SPA's JavaScript intercepts this successful HTTP response and dynamically alters the Document Object Model (DOM).

Rather than loading a heavy administrative dashboard, the SPA transitions to render a user-selected ASCII art interface. This ASCII art serves as a lightweight, visually distinct confirmation that the user has successfully breached the outer cryptographic perimeter. Once this interface is rendered, it acts as the interactive frontend, signaling that the loopback proxy is now actively authorizing and routing their traffic to the protected backend services.

Conclusion

Securing a loopback proxy demands the same architectural rigor, cryptographic discipline, and defense-in-depth methodologies as deploying an internet-facing perimeter gateway. By synthesizing the memory-safe Coraza WAF engine with the anomaly scoring of OWASP CRS v4, the proxy natively identifies and dismantles complex Layer 7 payloads and HTTP desynchronization vectors before they can impact backend logic. The integration of a kernel-level nftables killswitch provides an absolute, fail-closed defense against volumetric brute-forcing, shielding the computationally expensive Argon2id cryptographic operations from resource exhaustion.

Furthermore, by strictly adhering to the latest 2026 NDSS mitigations against JWT "Token Time Bombs" and locking session state behind unforgeable __Host- cookie prefixes, the architecture guarantees unparalleled session integrity. Ultimately, this framework ensures that the single page application—and the ASCII art it serves—remains accessible exclusively to fully verified, cryptographically authenticated users, establishing a formidable zero-trust boundary directly on port 1137.

Works cited

1. Build Network Proxies and Reverse Proxies in Go: A Hands-On Guide, https://dev.to/jones_charles_ad50858dbc0/build-network-proxies-and-reverse-proxies-in-go-a-hands-on-guide-288j
2. CVE-2026-77087: Paperclip RCE Vulnerability via DNS Rebinding, https://www.sentinelone.com/vulnerability-database/cve-2026-77087/
3. CVE-2026-61568: mcp-gitlab CSRF Vulnerability via DNS Rebinding, https://www.sentinelone.com/vulnerability-database/cve-2026-61568/
4. DNS Rebinding Attack: How Malicious Websites Exploit Private, https://unit42.paloaltonetworks.com/dns-rebinding/
5. CVE-2026-64849 Impact, Exploitability, and Mitigation Steps | Wiz, https://www.wiz.io/vulnerability-database/cve/cve-2026-64849
6. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3, https://www.rfc-editor.org/info/rfc8446/
7. TLS 1.3 in Node.js: 12 Steps to Secure HTTPS in 30 Min [2026], https://shattered.io/tls-1-3-nodejs/
8. How to make self-signed certificate for localhost?, https://unix.stackexchange.com/questions/288517/how-to-make-self-signed-certificate-for-localhost
9. Back to Basics: TLS and PKI from the Ground Up, https://www.msbiro.net/posts/back-to-basics-tls-pki/
10. OpenSSL Command Builder - RawOps, https://rawops.dev/tools/openssl-command-builder
11. Quick Start - OWASP Coraza, https://www.coraza.io/docs/tutorials/quick-start/
12. coraza package - github.com/jptosso/coraza-waf - Go Packages, https://pkg.go.dev/github.com/jptosso/coraza-waf
13. Introduction - OWASP Coraza, https://www.coraza.io/docs/tutorials/introduction/

14. WAF on a budget: protecting your app with Coraza + OWASP CRS, https://hogin.pro/posts/waf-minimal-self-hosted-coraza/
15. GitHub - AevonXApp/AXCerberus: High-performance Layer 7 Web, https://github.com/AevonXApp/AXCerberus
16. Web Application Protection Optimization Through Coraza WAF - MDPI, https://www.mdpi.com/2673-4591/115/1/17
17. How we built ngrok's WAF with OWASP CRS and Coraza | ngrok blog, https://ngrok.com/blog/ngrok-waf-owasp-crs-coraza
18. Smart WAF Deployment and Enhancing Web security using Bunker, https://medium.com/@adevani20045/smart-waf-deployment-and-enhancing-web-security-using-bunker-web-on-aws-6950c53f27bb
19. From Zero to Production-Grade WAF - Traefik Labs, https://traefik.io/blog/zero-to-production-grade-waf
20. CloFix WAF Feature | Web Application Firewall for Cloud Security, https://clofix.com/features
21. Protect Applications with Web Application Firewall - Traefik Labs, https://doc.traefik.io/traefik-hub/api-gateway/secure/middleware/waf-guide
22. CRS Documentation, https://coreruleset.org/docs/index.print
23. Evaluation Framework for False Positives in Open-Source WAFs, https://www.researchgate.net/publication/397675442_Evaluation_Framework_for_False_Positives_in_Open-Source_WAFs_Based_on_OWASP_CRS_Paranoia_Levels_A_Systematic_Approach_for_Comparative_Measurement
24. Quick Start - Coraza Proxy, https://alexperezortuno-go-coraza.mintlify.app/quickstart
25. Articles | systemshardening.com, https://www.systemshardening.com/articles/
26. coreruleset/CHANGES.md at main - GitHub, https://github.com/coreruleset/coreruleset/blob/main/CHANGES.md
27. Coraza Enterprise WAF & OWASP Core Rule Set (CRS v4, https://masacp.com/docs/coraza-enterprise-waf-owasp-core-rule-set-crs-v4-protection/
28. CrossLink: a Go proxy gateway for LLM APIs worth reading, https://www.bytesizego.com/blog/crosslink-a-go-proxy-gateway-for-llm-apis-worth-reading
29. The Rate Limiting Mistake That Cost Us a 4 am Call - Level Up Coding, https://levelup.gitconnected.com/the-rate-limiting-mistake-that-cost-us-a-4-am-call-d01659bfd31c
30. Web Application Firewall Fundamentals: Rules, Tuning, and the, https://veduis.com/blog/web-application-firewall-fundamentals/
31. VPN Kill Switch Demystified: How the OS Kernel Blocks Traffic and, https://vpn.how/en/pages/vpn-kill-switch-demystified-how-the-os-kernel-blocks-traffic-and-protects-your-privacy.html
32. Linux VPN Kill Switch: 7 Tests to Stop IP & DNS Leaks, https://hackersghost.com/linux-vpn-kill-switch/
33. How to Set Up a Router VPN Without Installing VPN Software, https://meshwg.com/blog/how-to-set-up-a-router-vpn-without-installing-vpn-s

oftware/
34. Linux Policy-Based Routing: Split Tunnel Like a Pro - VPN sites, https://vpnsites.com/post/linux-policy-based-routing-split-tunnel-like-a-pro/
35. Securing web applications with Coraza WAF and Wazuh, https://wazuh.com/blog/securing-web-applications-with-coraza-waf-and-wazuh/
36. PBKDF2 vs Argon2id: Password Hashing | MojoAuth Security Guides, https://mojoauth.com/security-guides/argon2-vs-pbkdf2
37. Picking a password hash: A developer's guide to argon2, bcrypt, and, https://workos.com/blog/picking-a-password-hash-argon2-bcrypt-scrypt
38. Password Hashing Algorithms in 2026: Argon2id, bcrypt & scrypt, https://bellatorcyber.com/blog/best-password-hashing-algorithms-of-2023
39. What Is a Password Hash? How It Works and How to Secure It, https://www.doppel.com/doppel-pedia/what-password-hash-how-it-works-and-how-secure-it
40. Best practices for password hashing and storage - IETF Datatracker, https://datatracker.ietf.org/doc/draft-ietf-kitten-password-storage/09/
41. Argon2 vs Bcrypt vs Scrypt vs PBKDF2 (2026 Guide) - Deepak Gupta, https://guptadeepak.com/the-complete-guide-to-password-hashing-argon2-vs-bcrypt-vs-scrypt-vs-pbkdf2-2026/
42. Argon2 vs Bcrypt vs Scrypt: One Needs 31x the RAM [2026], https://shattered.io/argon2-vs-bcrypt-vs-scrypt-2026/
43. RFC 9106: Argon2 Memory-Hard Function for Password Hashing, https://www.rfc-editor.org/info/rfc9106/
44. Argon2 in Golang: Hash Passwords with IDKey - MojoAuth, https://mojoauth.com/security-guides/argon2-in-go
45. Building authentication in Go applications: The complete guide for, https://workos.com/blog/go-authentication-guide
46. Secure Password Hashing in Go: bcrypt vs Argon2 - Safeguard.sh, https://safeguard.sh/resources/blog/secure-password-hashing-go
47. RFC 7519: JSON Web Token (JWT) - RFC Editor, https://www.rfc-editor.org/info/rfc7519/
48. penthera/docs/references.md at main - danoszz - GitHub, https://github.com/danoszz/penthera/blob/main/docs/references.md
49. Baosheng Wang (disambiguation) - dblp, https://dblp.org/pid/96/5513
50. NDSS-2026-Papers - GitHub, https://github.com/wzx442/NDSS-2026-Papers
51. Token Time Bomb: Evaluating JWT Implementations for Vulnerability, https://www.ndss-symposium.org/wp-content/uploads/2026-f697-paper.pdf
52. What Is Token-Based Authentication? JWT vs Session Explained, https://ambimat.com/what-is-token-based-authentication/
53. Cookie security: HttpOnly, Secure, SameSite, and the __Host- prefix, https://blog.crawlex.net/blog/cookie-security-attributes/
54. Cookie Chaos: How to bypass __Host and __Secure, https://portswigger.net/research/cookie-chaos-how-to-bypass-host-and-secure-cookie-prefixes

55. CVE-2024-2756: PHP Cookie Prefix Bypass Vulnerability, https://www.sentinelone.com/vulnerability-database/cve-2024-2756/
56. draft-ietf-httpbis-rfc6265bis-22, https://datatracker.ietf.org/doc/html/draft-ietf-httpbis-rfc6265bis-22
57. draft-ietf-httpbis-rfc6265bis-22 - Cookies: HTTP State Management, https://datatracker.ietf.org/doc/draft-ietf-httpbis-rfc6265bis/